UNIT I: INTRODUCTION TO ALGORITHMS AND MACHINE LEARNING

1. Introduction to Algorithms

An algorithm is a finite sequence of clear and well-defined steps used to solve a problem or perform a task. In simple words, an algorithm is a step-by-step procedure for getting the required output from given input.

1.1 Why Do We Need Algorithms?

- To solve problems systematically.
- To obtain correct and reliable results.
- To compare different solutions to the same problem.
- To estimate the time and memory required by a solution.
- To make programs easier to design, understand, test, and maintain.

1.2 Characteristics of a Good Algorithm

Characteristic	Meaning
Input	It accepts zero or more clearly specified inputs.
Output	It produces at least one result.
Definiteness	Every step must be clear and unambiguous.
Finiteness	The algorithm must terminate after a finite number of steps.
Effectiveness	Each operation should be basic enough to be carried out.
Correctness	It should produce the expected result for valid inputs.
Generality	It should solve a class of similar problems, not just one instance.

1.3 Algorithm vs Program

Algorithm	Program
Problem-solving procedure	Implementation of the procedure in a programming language
Language independent	Language dependent
Focuses on logic	Focuses on executable code
Can be written using pseudocode/flowchart	Written using syntax of a programming language

1.4 Ways to Represent an Algorithm

- Natural language: steps written in ordinary language.
- Pseudocode: structured English-like representation of logic.
- Flowchart: graphical representation using standard symbols.
- Programming language: executable implementation.

1.5 Example: Find the Largest of Two Numbers

1. Start.
2. Read A and B.
3. Compare A and B.
4. If A > B, display A as the largest.
5. Otherwise, display B as the largest.
6. Stop.

1.6 Example: Linear Search

Linear search checks each element one by one until the required element is found or the list ends.

7. Start from the first element.
8. Compare the current element with the target.
9. If they are equal, report the position.
10. Otherwise move to the next element.
11. If all elements are checked, report that the element is not present.

Time complexity: Best case O(1), worst case O(n), where n is the number of elements.

2. Tools to Analyze Algorithms

Algorithm analysis is the process of estimating the resources required by an algorithm, mainly execution time and memory, as the input size grows.

2.1 Input Size

Input size, usually represented by n, is the amount of data given to an algorithm. For example, in sorting n numbers, n is the number of elements.

2.2 Time Complexity

Time complexity describes how the number of basic operations performed by an algorithm grows with input size. It does not normally mean exact clock time; it gives a growth-rate estimate.

Notation	Meaning	Example
O(1)	Constant	Accessing an array element by index
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search
O(n log n)	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Bubble sort in worst case
$O(2^n)$	Exponential	Many brute-force subset algorithms
O(n!)	Factorial	Brute-force permutation generation

2.3 Space Complexity

Space complexity measures the additional memory required by an algorithm as the input size grows. It may include temporary variables, data structures, and recursion stack space.

2.4 Best, Average and Worst Case

Case	Meaning
Best case	Minimum resources required for an input of size n.
Average case	Expected resources over inputs according to an assumed distribution.
Worst case	Maximum resources required for an input of size n.

2.5 Asymptotic Notations

Asymptotic notation describes the growth of an algorithm for large input sizes while ignoring constants and lower-order terms.

Notation	Purpose	Simple Interpretation
Big-O, O(g(n))	Upper bound	Algorithm grows no faster than the stated order, asymptotically.
Big-Omega, Ω(g(n))	Lower bound	Algorithm grows at least as fast as the stated order, asymptotically.
Big-Theta, Θ(g(n))	Tight bound	Algorithm grows at the same order as the stated function.

2.6 Counting Basic Operations

One simple analysis technique is to count how many times an important operation executes. For example, a loop that runs n times and performs one comparison per iteration performs approximately n comparisons, so its time complexity is O(n).

2.7 Recurrence Relations

Recursive algorithms are often analyzed using recurrence relations. A recurrence expresses the running time of a problem in terms of smaller instances.

Example: Merge sort: $T(n) = 2T(n/2) + O(n)$. The two recursive calls handle two halves and O(n) work is used for merging.

2.8 Master Theorem – Basic Idea

The Master Theorem provides standard rules for solving recurrences of the form $T(n) = aT(n/b) + f(n)$. It is especially useful for divide-and-conquer algorithms.

- a = number of recursive subproblems.
- n/b = size of each subproblem.
- f(n) = work done outside recursive calls.

2.9 Factors Affecting Algorithm Performance

- Input size.
- Choice of data structure.
- Number of operations.
- Recursive calls.
- Memory usage and cache behavior.
- Quality of the algorithm and implementation.

3. Algorithmic Techniques

Algorithmic techniques are general strategies used to design efficient algorithms. Important techniques in this unit include Divide and Conquer and Randomization.

3.1 Divide and Conquer

Divide and Conquer solves a problem by dividing it into smaller subproblems, solving those subproblems, and combining their solutions.

3.1.1 Three Main Steps

12. Divide: Split the original problem into smaller subproblems.
13. Conquer: Solve each subproblem, usually recursively.
14. Combine: Combine the solutions of the subproblems to obtain the solution to the original problem.

3.1.2 General Structure

Problem → Divide → Smaller Problems → Solve Recursively → Combine → Final Solution

3.1.3 When is Divide and Conquer Useful?

- When a problem can be divided into smaller versions of the same problem.
- When subproblems can be solved independently or mostly independently.
- When combining the subproblem results is efficient.
- When recursion naturally matches the problem structure.

4. Examples of Divide and Conquer

4.1 Binary Search

Binary search works on a sorted list. It compares the target with the middle element and discards half of the remaining search space after each comparison.

15. Find the middle element.
16. If the middle element equals the target, stop.
17. If the target is smaller, search the left half.
18. If the target is larger, search the right half.
19. Repeat until the target is found or the search range becomes empty.

Time complexity: O(log n).

4.2 Merge Sort

Merge sort divides an array into two halves, recursively sorts each half, and then merges the two sorted halves.

20. Divide the array into two halves.
21. Continue dividing until each part contains one element.
22. Merge two sorted parts to create a larger sorted part.
23. Continue merging until the complete array is sorted.

Time complexity: O(n log n) in best, average, and worst cases. Extra space is typically O(n).

4.3 Quick Sort

Quick sort selects a pivot, partitions the elements around the pivot, and recursively sorts the two resulting parts.

24. Choose a pivot.
25. Place smaller elements on one side and larger elements on the other side according to the partition scheme.
26. Recursively sort the left and right portions.
27. The final arrangement is sorted.

Average time complexity: O(n log n). Worst case: $O(n^2)$, for example with consistently poor pivot choices.

4.4 Maximum and Minimum Using Divide and Conquer

The array can be divided into two parts. Find the maximum and minimum in each part recursively, then compare the two maxima and two minima. This demonstrates how a larger problem can be solved from smaller results.

4.5 Advantages and Limitations

Advantages	Limitations
Can reduce problem size rapidly	Recursion may use additional stack space
Often leads to efficient algorithms	Combining solutions may be costly
Naturally supports parallel processing in some problems	Not every problem can be divided effectively
Provides clean problem decomposition	Implementation can be more complex than a simple iterative method

5. Randomization in Algorithms

A randomized algorithm uses random choices during execution. The random choice can influence the path or operations performed by the algorithm, often helping avoid consistently bad behavior.

5.1 Why Use Randomization?

- To avoid predictable worst-case patterns.
- To simplify some algorithm designs.
- To improve expected performance.
- To make an algorithm less dependent on a particular input arrangement.

5.2 Randomized Quick Sort

In randomized quick sort, the pivot is selected randomly instead of always choosing the first, last, or a fixed position. This makes consistently poor pivot selection less likely for many input patterns.

Expected time complexity is O(n log n), while the worst case remains $O(n^2)$.

5.3 Types of Randomized Algorithms

Type	Idea
Las Vegas	Always produces a correct answer, but running time may vary.

Monte Carlo	Runs within a controlled/expected time or resource limit but may have a probability of error.

5.4 Real-World Examples

- Randomized search and sampling.
- Load balancing and randomized selection.
- Cryptographic and security algorithms.
- Network protocols and distributed systems.
- Randomized optimization and machine learning methods.
- Randomized pivot selection in sorting.

5.5 Simple Example

Suppose a teacher randomly selects one student from a class to answer a question. The selected student changes each time. Similarly, a randomized algorithm makes a random choice to influence its execution path.

5.6 Advantages and Limitations

Advantages	**Limitations**
Can avoid predictable bad inputs	Results or execution path may vary
Often simple to implement	Correctness/performance may require probability-based analysis
Useful for large and complex problems	Random-number generation introduces overhead
Can improve expected performance	Worst-case complexity may still exist

6. Applications of Algorithms

Algorithms are the basic problem-solving mechanisms behind software systems, AI, data science, networks, operating systems, and everyday digital services.

Area	**Example of Algorithmic Use**
Search engines	Ranking, indexing, query processing
Navigation	Shortest-path and route-finding algorithms
Social media	Recommendation, ranking, graph algorithms
E-commerce	Product recommendation, search, price analysis
Banking	Fraud detection, transaction processing, risk analysis
Healthcare	Medical image analysis, diagnosis support, scheduling
Cybersecurity	Encryption, authentication, anomaly detection
Operating systems	Scheduling, memory management, resource allocation
Networks	Routing and congestion control
Artificial Intelligence	Search, planning, game playing, inference
Machine Learning	Training, optimization, classification, clustering
Databases	Searching, indexing, sorting, query optimization

7. Introduction to Machine Learning

Machine Learning (ML) is a branch of Artificial Intelligence in which computers learn patterns from data and use those patterns to make predictions, decisions, or classifications without being explicitly programmed with every rule.

7.1 Traditional Programming vs Machine Learning

Traditional Programming	Machine Learning
Rules + Data → Output	Data + Expected examples → Learned model
Rules are explicitly written by programmer	Model learns patterns from data
Works well when rules are known	Useful when rules are difficult to write explicitly

7.2 Main Types of Machine Learning

Type	Basic Idea	Example
Supervised Learning	Learns from labeled data	Spam/not-spam classification
Unsupervised Learning	Finds patterns in unlabeled data	Customer clustering
Reinforcement Learning	Learns through rewards and penalties	Game-playing agent or robot control

7.3 Basic Machine Learning Workflow

28. Define the problem.
29. Collect relevant data.
30. Clean and prepare the data.
31. Select useful features or representations.
32. Split data into training and testing sets when appropriate.
33. Choose a suitable model.
34. Train the model.
35. Evaluate its performance.
36. Tune or improve the model.
37. Deploy and monitor it.

7.4 Important Terms

Term	Meaning
Dataset	Collection of data used for analysis or learning.
Feature	Input characteristic used by a model.
Label	Known target/output in supervised learning.
Model	Learned representation used to make predictions.
Training	Process of learning model parameters from data.
Testing	Evaluation on data not used for training.
Prediction	Output generated by a trained model.

7.5 Algorithms and Machine Learning – Connection

Machine learning itself depends on algorithms. Algorithms are used to preprocess data, train models, optimize parameters, search for solutions, classify observations, cluster data, and evaluate models. Therefore, efficient algorithm design is important for practical machine learning.

7.6 Examples of Machine Learning Applications

- Email spam detection.

- Face and image recognition.
- Recommendation systems.
- Speech recognition.
- Medical diagnosis support.
- Fraud detection.
- Predictive maintenance.
- Customer segmentation.
- Autonomous vehicles and robots.
- Text classification and sentiment analysis.

8. Quick Comparison of Major Concepts

Concept	Core Idea	Typical Example
Algorithm	Step-by-step solution	Linear search
Algorithm Analysis	Measure resource growth	Time/space complexity
Divide and Conquer	Divide, solve, combine	Merge sort
Randomization	Use random choices	Randomized quick sort
Machine Learning	Learn patterns from data	Spam classification